



GROUP 2

Ho Chi Minh City Transportation Platform Progress Report

Progress Report

November 2025

Contents

1	Introduction	1
1.1	Problem Definition	1
1.2	Team Roles Overview	1
2	Routing	3
2.1	Introduction	3
2.1.1	Problem Definition	3
2.1.2	Solution Approach	3
2.1.3	Framework & Tech Stack	4
2.2	Implemented Tasks	4
2.2.1	Foundation & Configuration	4
2.2.2	Core Routing API Development	5
2.2.3	Search API Development	5
2.2.4	Route History Management	5
2.2.5	Error Handling	5
2.3	Next Steps	6
2.4	Summary	6
3	Bus Map System	7
3.1	Introduction	7
3.1.1	Problem Definition	7
3.1.2	Stakeholders	7
3.1.3	Objectives	8
3.1.4	System Inputs	8
3.1.5	System Outputs	8
3.1.6	System Constraints	8
3.2	Implemented Tasks	9
3.2.1	Data Layer Development	9
3.2.2	Pathfinding Algorithm Implementation	9
3.2.3	Realistic Modeling Features	10
3.2.4	Backend API Development	10
3.2.5	Map Generation and Export	11
3.2.6	Tools and Technologies	11
3.3	Next Steps	11
3.4	Summary	12
4	Traffic Congestion	13
4.1	Introduction	13
4.1.1	Problem Definition	13
4.1.2	Proposed Solution	13
4.1.3	System Architecture Framework	14

4.2	Implemented Tasks	14
4.2.1	Vehicle Detection System	14
4.2.2	Data Aggregation Layer	14
4.2.3	REST API Development	15
4.2.4	Web Frontend	15
4.2.5	Technology Stack	16
4.3	Next Steps	16
4.4	Summary	17
5	Data Collection	18
5.1	Introduction	18
5.1.1	Module Goals	18
5.2	Implemented Tasks	18
5.2.1	Data Collection Pipeline	18
5.2.2	Real-Time Frame Scraping	19
5.2.3	Frame Extraction & Preprocessing	19
5.2.4	Local and Cloud Storage Management	19
5.2.5	Privacy and Compliance	19
5.2.6	Monitoring & Observability	20
5.2.7	API Endpoints	20
5.3	Next Steps	20
5.3.1	Advanced Privacy Controls	20
5.3.2	Cloud Integration	20
5.3.3	Calibration & Spatial Understanding	21
5.3.4	Stability & Scaling Improvements	21
5.3.5	Dashboard & Alerting	21
5.3.6	Integration with Other Modules	21
5.4	Summary	21
6	Database	23
6.1	Introduction	23
6.2	Implemented Tasks	23
6.3	Next Steps	24
6.4	Summary	24
7	UX/UI Integration	25
7.1	Introduction	25
7.2	Implemented Tasks	25
7.2.1	Authentication & OAuth UI	25
7.2.2	Map & Routing Interfaces	26
7.2.3	Styling & Design System	26
7.2.4	App Structure & Navigation	26
7.3	Next Steps	26
7.4	Summary	27

Chapter 1

Introduction

1.1 Problem Definition

Travelers, from leisure visitors to business professionals, face significant and costly friction in planning and executing urban itineraries, which stems from a core problem of fragmented data. Critical decision-making data on public transit schedules (GTFS), ride-hail fares, live traffic, and operating hours exists in disconnected silos.

This forces users to manually compare multiple apps, leading to:

- Inefficient planning under complex constraints (time, budget, transfers).
- Suboptimal in-the-moment transport choices, failing to balance cost, ETA.
- Reactive, unactionable responses to congestion and crowding.

1.2 Team Roles Overview

Role	Member	Key Responsibility
Routing & SOS Map	Nguyễn Thành Đạt (ID: 24127021)	TomTom API integration, Search API Development, Route History Management
Bus Map System	Hoàng Trung Hiếu (ID: 24127041)	Graph assembly and modeling, navigational pathfinding computation, identification and retrieval of bus itineraries
Safety & Monitoring	Lương Hưng Phát (ID: 24127298)	Real-time congestion detection, alert rule calibration
Data Collection	Nguyễn Minh Khoa (ID: 24127188)	Scrape real-time feeds, normalization, validation checks
Database	Nguyễn Anh Khôi (ID: 24127430)	Schema design, indexing, backup & retention strategy
UI/UX & Integration	Nguyễn Tấn Hiệu (ID: 24127373)	Frontend implementation, cross-module API integration

Table 1.1: Team roles and ownership

Chapter 2

Routing

2.1 Introduction

2.1.1 Problem Definition

Goal: Build a function to search for and display the shortest/fastest route between two points within the scope of Ho Chi Minh City. **Specific Requirements:**

- **Input:** Users provide addresses or select coordinates on the map for the origin and destination.
- **Processing:**
 - Convert addresses to coordinates (Geocoding)
 - Calculate the route based on mode of transport (car, bicycle, pedestrian) and criteria (fastest, shortest)
 - Return a list of coordinates (polyline) for the route, total distance (km), and estimated duration (minutes)
- **Output:** Provide clean JSON data for the Frontend to render the route on Goong Maps (JS SDK) and display details.
- **Constraints:** The Backend system (Flask) acts as a Proxy Service calling the Goong Directions API to fetch data, then processing and returning it to the Frontend (React).

2.1.2 Solution Approach

The Backend system applies a Proxy Service model combined with Geofencing Middleware:

- **Business Logic Processing:**
 - Create a central endpoint `/route` to handle all routing requests
 - **Geofencing:** Before making calls to third-party APIs, the system validates whether the start/end coordinates fall within the Bounding Box of Ho Chi Minh City (Min/Max Lat/Lon)

- **Data Parsing:** The Backend receives raw, complex JSON from TomTom and extracts only necessary data (arrays of lat/lon coordinates, total distance, total time) to minimize the payload sent to the client
- **Service Integration:** Use the Python `requests` library to perform Server-to-Server communication with the Goong Maps Directions / Geocoding APIs.
- **Security & Personalization:** Use JWT Middleware (via the `token_required` decorator) to protect route history storage APIs (`/api/user/routes`), ensuring users can only access their own data.

2.1.3 Framework & Tech Stack

- **Core Framework:**
 - **Python (Flask):** Main framework for building the API server
 - **Flask-CORS:** Handles Cross-Origin Resource Sharing, allowing the React Frontend to consume APIs
- **Database Integration:**
 - **MySQL Connector:** Library for connecting to the database to execute SQL queries for storing `user_routes`
- **Security:**
 - **PyJWT:** Generates and validates login tokens
 - **Bcrypt:** Hashes user passwords for security
- **External Service:**
 - **Goong Maps Directions API:** Third-party service for route calculation (fastest/shortest with traffic)

2.2 Implemented Tasks

The routing module development has progressed through five distinct phases:

2.2.1 Foundation & Configuration

- Initialized the Flask Server and configured MySQL connection (`get_db_connection`)
- Set up environment variables and critical constants: `GOONG_API_KEY` and `HCM_BBOX` (coordinate limits for HCMC)
- Configured `CORS(app)` to accept requests from all sources in the development environment

2.2.2 Core Routing API Development

Built Endpoint POST `/route` with the following features:

- **Input Validation:** Check for the existence of `start` and `end` in the request body
- **Geofencing Check:** Implemented the `is_in_hcm(lat, lon)` function to validate coordinates. If coordinates are outside the HCMC boundary, return a 400 error immediately to prevent wasteful API calls to Goong
- **API Proxying:** Configured the request sent to Goong Directions with parameters (e.g., traffic-aware, fastest route type)
- **Response Formatting:** Transformed the complex TomTom structure into a simple JSON format: `{coords: [{lat, lon}...], distance_km, duration_min}`

2.2.3 Search API Development

Built Endpoint POST `/search` to support routing:

- Handles logic for location searching to obtain input coordinates for routing
- Integrated parameters `countrySet=VN` and `radius` to limit search results to the area surrounding the HCMC center, improving accuracy for local users

2.2.4 Route History Management

Implemented user history management features:

- Built Decorator `@token_required`: Middleware to block requests missing the `Authorization: Bearer ...` header or containing invalid tokens
- Built Endpoint POST `/api/user/routes`:
 - Receives route information (`start_name, end_name, coordinates`)
 - Executes the SQL `INSERT INTO user_routes` command associated with the `user_id` decoded from the token
- Built Endpoint GET `/api/user/routes`:
 - Executes SQL query `SELECT * FROM user_routes` filtering by the current `user_id`, sorted by the latest time (`ORDER BY timestamp DESC`)

2.2.5 Error Handling

- Caught connection exceptions (`requests.exceptions`) when calling Goong APIs to prevent server crashes
- Implemented standard HTTP Status Codes:
 - **400** (Bad Request - invalid coordinates)
 - **401** (Unauthorized - missing token)
 - **500** (Internal Server Error - third-party API failure)

2.3 Next Steps

The following tasks are planned for future development:

- **Alternative Routes:** Implement k-shortest paths algorithm to provide users with multiple route options ranked by different criteria (fastest, shortest, least congested, most scenic)
- **Enhanced Multi-modal Support:** Expand transportation mode options beyond car, bicycle, and pedestrian to include motorcycle and public transit integration
- **Performance Optimization:** Implement caching strategies for frequently requested routes and optimize API response times through connection pooling and async processing
- **Advanced Geofencing:** Extend geofencing capabilities to support district-level filtering and custom boundary definitions for specialized routing scenarios

2.4 Summary

The routing module has established a robust proxy architecture integrating a Flask backend with the Google Maps Directions and Geocoding APIs. It delivers route calculation with geofencing validation, multi-modal transport support, and secure user history management. The five development phases (foundation, core routing, search, history, error handling) are complete; the system now supports traffic-aware routing and returns minimal, frontend-optimized JSON payloads. Future work targets k-shortest alternative paths, expanded modal coverage, and performance improvements (caching, async I/O) for city-scale usage.

Chapter 3

Bus Map System

3.1 Introduction

This chapter describes the development of the BusMap module, which provides an independent system for public transportation routing in Ho Chi Minh City. Public transportation plays an essential role in urban mobility, with hundreds of bus routes covering both central and suburban districts, making buses one of the most affordable travel options for citizens and tourists.

3.1.1 Problem Definition

While public transportation in Ho Chi Minh City is widely available, it remains difficult for first-time visitors to navigate efficiently. Existing mobile applications (e.g., BusMap, Google Maps) provide bus route guidance, but their proprietary algorithms lack transparency and are not easily extensible for research-driven experimentation.

This project aims to build an **independent system** that helps visitors choose the **optimal bus route**, considering **travel time, transfers, cost, and waiting penalties**. A key challenge is the **lack of open-source and up-to-date bus route data**. To address this, data was collected manually from official city transport sources and combined into structured CSV files, forming the foundational dataset of this project.

3.1.2 Stakeholders

- **Visitors / Tourists:** Need clear, reliable, and optimized bus routes between two locations
- **Local Developers / Researchers:** Require a reusable dataset and routing engine for transport-related studies
- **City Planners (potential):** Could use the model to analyze route efficiency or passenger flow

3.1.3 Objectives

- Build a data-driven routing system that suggests **optimal bus paths** between two coordinates in HCMC
- Model **realistic travel conditions**, including transfer costs and waiting times
- Provide a **scalable graph-based framework** that can later integrate live traffic or timetable updates

3.1.4 System Inputs

User Input:

- Origin coordinate (latitude, longitude)
- Destination coordinate (latitude, longitude)
- MaxWalkingDistance (meter) — **default = 400 m**: the maximum distance the user is willing to walk

Dataset Files:

- `stops.csv` — list of bus stops with coordinates
- `routes.csv` — metadata for each bus route (ID, name, fare, active time, spacing penalty)
- `trips.csv` — ordered sequence of stops for each route and distance between them

3.1.5 System Outputs

- The **optimal route** (sequence of bus stops and route IDs) from start to destination
- The **estimated travel distance or time**, including waiting and transfer penalties
- The **total fares** of the trip
- A **visualized route** on an interactive web map (Goong Maps JS SDK)

3.1.6 System Constraints

- **No official open-source bus dataset** is available — data must be collected manually
- **Static scheduling** — the system initially assumes average bus frequency, not live tracking
- **Simplified cost model** — considers distance and waiting penalties but not fare differences or road congestion
- **Limited accuracy in spacing data** — since BusMap-provided "bus stop spacing" may be approximate or in ranges (e.g., "50–55")

3.2 Implemented Tasks

The BusMap module has progressed through multiple development phases:

3.2.1 Data Layer Development

Input File Structure:

- **stops.csv**: Contains **stop_id**, **name**, and geographical coordinates (latitude, longitude)
- **routes.csv**: Contains **route_id**, **name**, and metadata such as average bus spacing or waiting time
- **trips.csv**: Lists all bus stops for each route in sequential order, defining the structure of each bus trip

Graph Construction:

- Each bus stop is treated as a **node**
- Adjacent stops on the same route are connected by **directed edges**
- Edge weights represent **travel distance** calculated using the Haversine formula and stored in **routes.csv**
- All graphs are constructed initially and stored in cache throughout server runtime for optimal performance

3.2.2 Pathfinding Algorithm Implementation

Implemented a modified **A*** search algorithm that considers:

- Distance between stops
- Transfer penalties between routes
- Starting penalty for initial waiting time
- Bus fare costs

Cost Function:

$$\text{cost} = \text{currentcost} + \frac{\text{edgeDist}}{v} + \text{transferPen} \cdot p + c \cdot \text{fare}$$

Where:

- **currentcost** — accumulated cost to reach the current node
- **edgeDist** — distance (in meters) of the current bus segment

- **transferPen** — transfer penalty, equal to the average bus spacing / waiting time; set to 0 if there's no transfer between routes
- **v (m/s)** — average bus speed, assumed to be **7 m/s**
- **p** — constant coefficient that amplifies the importance of bus transfer penalties
- **c** — constant coefficient that amplifies the importance of bus fare
- **fare** — bus fare cost; fare = 0 if there's no route transfer

Duration Estimation:

$$\text{WorstCaseDuration} = \text{TotalCost} - c \cdot \text{TotalFares} - (p - 1) \cdot \text{TotalWait} + \frac{\text{walkDistance}}{\text{walkSpeed}}$$

$$\text{BestCaseDuration} = \text{WorstCaseDuration} - \text{TotalWait}$$

The algorithm compares multiple potential starting stops within **MaxWalkingDistance** range and selects the route with the lowest total cost.

3.2.3 Realistic Modeling Features

- **Transfer Penalty:** Adds time when changing from one route to another, reflecting bus waiting time
- **Starting Penalty:** Models the time a visitor might wait for the first bus to arrive
- **Heuristic Function:** Uses geographical distance (via Haversine formula) to guide the A* search efficiently

3.2.4 Backend API Development

- Built **Flask-based API** (`API.py`) to handle bus routing requests
- Implemented in-memory **graph_cache** system for efficient graph storage and retrieval
- Integrated **MySQL database** (XAMPP) for persistent storage of user-related data (future expansion)
- Connected with **Goong Geocoding API** for address → coordinate resolution (replacing earlier TomTom dependency)
- Developed **ranking logic** to prioritize optimal bus paths based on composite cost (distance, transfers, fares)

3.2.5 Map Generation and Export

- Backend collects routing data and sends JSON response to frontend
- Frontend (`GoogMap.js` / `GoogBusMap.js`) to render, draw this data into a standalone interactive `.html` format.
- JSON format includes coordinates and routing information: `{coords, routingInfo}`
- `GoogMap.js` / `GoogBusMap.js` creates HTML format output
- Interactive maps are displayed on `BusMapPage.js` / `RoutesPage.js`

3.2.6 Tools and Technologies

Category	Tool / Library	Purpose
Language	Python	Main implementation language
Data Handling	pandas, csv	Reading and managing bus stop and route data
Graph Algorithms	Custom A* & Dijkstra	Finding optimal routes with custom penalties
Math & Geo Utilities	math, Haversine formula	Computing real-world distances between coordinates
Visualization	Goong Maps JS SDK	Plotting routes and stops on interactive map
API Service	Goong Geocoding/Directions	Converting addresses to coordinates / future live directions
Backend Framework	Flask	API service implementation
Database	XAMPP (MySQL)	Persistent data storage
Documentation	Markdown	Project reports and technical documentation

3.3 Next Steps

Future development priorities for the BusMap module include:

- **Real-time Bus Tracking:** Integrate live GPS data to display current bus positions and real-time arrival predictions
- **Enhanced Cost Model:** Incorporate road congestion data, time-of-day variations, and detailed fare structures for more accurate routing
- **API Documentation:** Create comprehensive API documentation for third-party developers to integrate the routing service
- **Performance Optimization:** Implement advanced caching strategies and optimize A* heuristics for faster route computation

3.4 Summary

The BusMap module has established an independent, data-driven public transportation routing system for Ho Chi Minh City. It addresses the challenge of limited open-source bus data through manual collection and structuring into normalized CSV layers (stops, routes, trips). A graph-based routing engine (modified A*) incorporates transfer penalties, initial waiting time, walking distance constraints, and fare weighting to produce realistic itineraries. The current implementation includes a Flask backend, in-memory graph cache, MySQL integration foundation, Goong Maps geocoding support, and interactive frontend visualization. This transparent design enables experimentation, refinement of cost models, and future integration of live timetable or congestion data. The system already delivers optimal bus route recommendations with interpretable cost components for both visitors and research use cases.

Chapter 4

Traffic Congestion

4.1 Introduction

This chapter presents the development of the traffic congestion module, which addresses one of Ho Chi Minh City's most pressing urban challenges: endless traffic congestion, especially during rush hours. To help commuters avoid traffic jams, we propose a traffic-aware routing feature that leverages real-time traffic camera feeds to analyze current traffic conditions and suggest alternative routes.

The system processes live traffic data from multiple camera sources to identify congestion hotspots, assess severity levels, and provide actionable insights for route optimization and informed travel decisions.

The underlying camera ingestion and preprocessing pipeline is documented in detail in Chapter 5 (Data Collection).

4.1.1 Problem Definition

Ho Chi Minh City experiences severe and persistent traffic congestion, particularly during peak commuting hours. This congestion:

- Causes significant delays and unpredictable travel times
- Reduces overall mobility and economic productivity
- Creates frustration for daily commuters and visitors
- Lacks real-time visibility for route planning decisions

4.1.2 Proposed Solution

To address these challenges, we developed a **traffic-aware routing feature** that:

- Leverages real-time traffic camera feeds from across the city
- Analyzes current traffic conditions using computer vision
- Detects vehicle density and traffic flow patterns

- Suggests alternative routes to avoid congested areas
- Provides up-to-date traffic information to commuters

4.1.3 System Architecture Framework

The traffic congestion system follows a multi-stage pipeline architecture:

1. **Data Collection Pipeline:** Multi-threaded crawler continuously fetches traffic camera images and stores them in MySQL database
2. **Vehicle Detection System:** YOLOv11-based computer vision model analyzes images to detect and count vehicles
3. **Data Aggregation Layer:** Processes detection results to compute congestion metrics and traffic density levels
4. **REST API (Flask):** Provides access to processed traffic data and congestion information for client applications
5. **Web Frontend:** Displays real-time traffic conditions and integrates with routing features

4.2 Implemented Tasks

4.2.1 Vehicle Detection System

- Integrated **YOLOv11** object detection model
- Pretrained model for detecting various vehicle types (cars, motorcycles, buses, trucks)
- Achieved high detection accuracy suitable for real-time traffic analysis
- Implemented batch processing capabilities for efficient image analysis
- Optimized inference speed to handle high-frequency camera feeds
- Built vehicle counting and tracking logic to assess traffic density

4.2.2 Data Aggregation Layer

- Developed algorithms to aggregate vehicle detection results across multiple camera locations
- Implemented **congestion level classification** based on vehicle density thresholds
- Created time-series data structures for tracking congestion patterns over time
- Built spatial aggregation to compute area-wide traffic conditions
- Implemented statistical analysis for identifying traffic trends and anomalies
- Designed efficient data structures for fast query and retrieval

4.2.3 REST API Development

Completed Components:

- Built **Flask-based REST API** endpoints for accessing traffic data
- Implemented endpoints for current congestion status by location
- Created API for historical traffic pattern queries
- Developed endpoints for camera image retrieval and metadata
- Implemented authentication and rate limiting for API security
- Created comprehensive API response formatting with JSON schemas

Ongoing Work:

- **Testing integration with frontend** to ensure seamless data flow
- **Building real-time server container** for the YOLOv11 model to enable live processing
- Optimizing API response times for high-traffic scenarios
- Implementing WebSocket support for push-based real-time updates

4.2.4 Web Frontend

Completed Components:

- Developed base UI components for traffic visualization
- Implemented map integration displaying traffic camera locations
- Created traffic condition overlay with color-coded severity indicators
- Built basic route suggestion interface with congestion awareness
- Implemented user preference settings for traffic alerts

Ongoing Work:

- Continuous editing and refinement of user interface elements
- Enhancing real-time update mechanisms for live traffic data
- Improving responsive design for mobile device compatibility
- Adding interactive features for exploring traffic patterns
- Integrating with routing module for comprehensive route planning

4.2.5 Technology Stack

- **Programming Language:** Python
- **Web Framework:** Flask (REST API server)
- **Computer Vision:** YOLOv11 (vehicle detection)
- **Database:** MySQL (image and metadata storage)
- **Concurrency:** Multi-threading (parallel data collection)
- **Frontend:** React (web interface)
- **Containerization:** Docker (model deployment)

4.3 Next Steps

Future development priorities include:

- **Complete REST API Integration:**
 - Finalize frontend integration testing
 - Deploy production-ready real-time server container
 - Implement comprehensive error handling and monitoring
- **Complete Web Frontend:**
 - Polish user interface and user experience
 - Complete mobile responsiveness optimization
 - Finalize integration with routing and map features
- **Predictive Analytics:** Implement machine learning models to forecast traffic conditions 15-60 minutes ahead based on historical patterns
- **Incident Detection:** Develop anomaly detection to identify accidents, breakdowns, or unusual congestion patterns automatically
- **Performance Optimization:** Optimize model inference speed and API response times for scalability to city-wide deployment
- **Alert System:** Build notification service to alert users about sudden traffic changes or congestion on saved routes
- **Historical Analysis Dashboard:** Create analytics tools for studying long-term traffic trends and patterns
- **Multi-camera Fusion:** Enhance spatial aggregation to provide comprehensive traffic view across connected road segments

4.4 Summary

The traffic congestion module has made substantial progress in addressing Ho Chi Minh City’s traffic challenges through a computer vision-driven architecture. It builds on the multi-threaded ingestion foundation described in Chapter 5, combining: (1) the completed camera data pipeline (100%), (2) the YOLOv11 vehicle detection system (100%), and (3) the aggregation layer (100%). The REST API is 70% complete (focus: real-time containerization and frontend integration) and the web frontend stands at 60% (UI refinement and interactive exploration). This traffic-aware system converts raw camera frames into actionable congestion intelligence for routing decisions and user situational awareness, positioning the platform to improve urban mobility through proactive avoidance of emerging hotspots.

Chapter 5

Data Collection

5.1 Introduction

The Data Collection module is responsible for acquiring continuous, high-quality, and legally compliant real-time images from traffic surveillance cameras. These images serve as the foundation for downstream transportation services such as routing, bus map visualization, SOS incident detection, congestion scoring, and machine-learning-based flow estimation.

Real-world constraints make data ingestion challenging. Camera sources vary in resolution, frame rate, connectivity stability, and legal permissions. Additionally, environmental factors (lighting, weather, occlusion) may degrade image clarity. The objective of this module is to guarantee a robust, low-latency pipeline that captures clean frames reliably and consistently.

5.1.1 Module Goals

The goal of the Data Collection module is to:

- Maintain stable ingestion of real-time camera snapshots or streams
- Normalize and preprocess frames for downstream models
- Enforce compliance and privacy constraints
- Expose APIs for accessing frames and camera health
- Provide monitoring to ensure operational reliability

5.2 Implemented Tasks

5.2.1 Data Collection Pipeline

- Implemented **multi-threaded crawler** to fetch traffic camera images from `https://giaothong.hochiminhcity.gov.vn`
- Designed efficient database schema for **storing images in MySQL**

- Optimized storage strategy to balance image quality with database performance
- Implemented continuous data collection with configurable fetch intervals
- Built error handling and retry mechanisms for reliable data acquisition
- Established data retention policies to manage storage requirements

5.2.2 Real-Time Frame Scraping

Developed a scraper/fetcher capable of:

- **Continuous pulling of snapshots** from multiple camera sources
- **Retry logic and timeouts** for handling unstable connections
- **Adaptive rate limiting** to respect bandwidth and server constraints
- **Stream sampling via FFmpeg** for cameras with real video feeds
- Implemented **per-camera deduplication** using SHA-256 hashing to avoid storing identical frames

5.2.3 Frame Extraction & Preprocessing

Pipeline includes:

- **Uniform resizing** (1280×720) for standardized processing
- **Contrast correction** via YUV histogram equalization
- **Optional ROI masking** for narrowing the visible region
- **JPEG encoding** with quality controls

These steps standardize frames for congestion detection and object detection modules.

5.2.4 Local and Cloud Storage Management

- Created a **structured storage folder** per camera ID
- Implemented **rolling retention** (24–72 hours) with automatic deletion of old frames
- Optimized storage organization for efficient retrieval

5.2.5 Privacy and Compliance

- Configurable rules per camera:
 - Allowed/blocked usage
 - Optional masking of specific areas
- Integration-ready for face/plate blurring
- Ensured adherence to relevant camera usage terms

5.2.6 Monitoring & Observability

- Exposed **Prometheus metrics** including:
 - Frame freshness
 - Fetch success/failure counts
 - Camera uptime
 - Image storage rate
- Built foundations for **Grafana dashboards** for:
 - Camera health
 - Latency and stability
 - Error bursts

5.2.7 API Endpoints

A lightweight **FastAPI service** provides:

- `/cameras` — list of active camera sources
- `/camera/{id}/latest` — latest processed frame
- `/camera/{id}/health` — metadata + scraper statistics

This enables integration with downstream UI systems and ML pipelines.

5.3 Next Steps

5.3.1 Advanced Privacy Controls

- Add automated face and license plate blurring using YOLO/RetinaFace + OpenCV
- Support camera-based privacy zones with configurable masking
- Implement comprehensive GDPR and local privacy law compliance features

5.3.2 Cloud Integration

- Generate presigned URLs for use across Routing, Bus Map, and SOS modules
- Implement backup and disaster recovery mechanisms
- Optimize cloud storage costs through intelligent archival policies

5.3.3 Calibration & Spatial Understanding

- Complete per-camera homography mapping to estimate real-world positions
- Prepare calibration files used by:
 - Flow estimation
 - Speed measurement
 - Congestion heatmap generation

5.3.4 Stability & Scaling Improvements

- Introduce asynchronous fetch loops (`aiohttp`) for lower latency
- Horizontal scaling using Docker & Kubernetes
- Automatic camera recovery for unstable endpoints
- Load balancing across multiple scraper instances

5.3.5 Dashboard & Alerting

- Finalize Grafana dashboards for:
 - Camera offline alerts
 - High failure rates
 - Latency spikes
- Integrate with Slack/Email alerting
- Implement real-time notification system for critical issues

5.3.6 Integration with Other Modules

Connect to:

- **Traffic Congestion Detection** — regular frame feed
- **Routing** — live congestion weight updates

5.4 Summary

The Data Collection module establishes a robust, real-time ingestion layer for traffic camera data—one of the most critical components of the transportation analytics system. The module currently supports stable scraping, preprocessing, standardized storage, monitoring, and basic compliance controls. This enables downstream systems to rely on a consistent and high-quality image stream.

Key accomplishments include comprehensive camera registry management, real-time frame scraping with deduplication, standardized preprocessing pipeline, rolling retention storage,

privacy compliance framework, Prometheus-based monitoring, and FastAPI endpoints for system integration. The module handles diverse camera sources with varying resolutions, frame rates, and connectivity stability while maintaining data quality through preprocessing and validation.

Upcoming work focuses on enhancing privacy protection through automated blurring, scaling performance with asynchronous operations and containerization, implementing spatial calibration for accurate measurements, integrating with cloud storage (S3/MinIO), and building comprehensive monitoring dashboards. Once completed, the module will become a fully production-ready data backbone, supporting advanced AI-driven traffic analytics across the entire platform and enabling seamless integration with routing, congestion detection, bus mapping, and incident management systems.

Chapter 6

Database

6.1 Introduction

This chapter presents the development of the database module, which provides the persistent storage infrastructure for the transportation data platform. The database system manages diverse data types including user/authentication records, SOS incidents, camera metadata and images, and bus routing datasets (stops, routes, trips), ensuring efficient storage, retrieval, and query performance.

A well-designed database architecture is fundamental to supporting analytics, real-time applications, and historical trend analysis across the platform.

6.2 Implemented Tasks

The database module has accomplished the following:

- **Database Platform:** Standardized on **MySQL** (XAMPP) for the application database used by the Flask backend.
- **Core Schemas:**
 - **users:** id, username, email (unique), password (nullable for OAuth), oauth_provider, oauth_provider_id, created_at
 - **user_locations:** id, user_id (FK), name, lat, lon, created_at
 - **user_routes:** id, user_id (FK), start_name, end_name, coords (JSON), distance_km, duration_min, created_at
 - **sos_locations:** id, user_id (FK), lat, lon, image_url, status, created_at
 - **camera_locations:** id, name, lat, lon, url, refresh_rate, status, created_at
 - **camera_frames** (optional): id, camera_id (FK), ts, storage_path, sha256
 - **bus_stops** (from stops.csv): stop_id, name, lat, lon
 - **bus_routes** (from routes.csv): route_id, name, fare, spacing_penalty
 - **bus_trips** (from trips.csv): route_id, seq, stop_id, dist_m
- **OAuth Migration:** Applied changes to support social login (add email unique, oauth_provider, oauth_provider_id; allow NULL password). Migration script: `temp_server/oauth_migration.sql`.

- **Indexing Strategy:** Created indexes on `users.email`, `user_locations.user_id`, `user_routes.user_id`, and geo-related columns (lat/lon where applicable) to optimize frequent queries.
- **Backup and Retention:** Established retention policies for camera frames (24–72h rolling) and scheduled MySQL dumps for application tables.
- **ETL Loading:** Implemented loading of CSV-based bus datasets into MySQL and caching in memory (`graph_cache`) for routing performance.

6.3 Next Steps

Future development priorities for the database module include:

- **Performance Tuning:** Analyze query plans, add composite indexes for common filters (e.g., `user_id + created_at`), and consider table partitioning for large append-only tables (frames, routes history).
- **Caching Layer:** Use Redis for hot-path reads (e.g., `user_routes` recent, bus graph metadata) to reduce DB load and improve latency.
- **Data Archival:** Implement S3/MinIO archival for camera frames beyond retention windows; keep metadata pointers for recall.
- **High Availability:** Set up MySQL replication and routine backups; add health checks and failover procedures.
- **Security Enhancements:** Enforce least-privilege DB users, at-rest encryption where possible, and secure connection strings via environment variables.

6.4 Summary

The database module has established a robust foundation using MySQL for core application data (auth, user content, SOS, camera registry/frames, and bus datasets). The system efficiently stores and retrieves diverse transportation data with proper indexing, retention/backup procedures, and data integrity constraints. Future work will focus on performance optimization, high availability, caching strategies, and archival integration. The database infrastructure is well-positioned to scale with the platform's growing data needs and analytical requirements.

Chapter 7

UX/UI Integration

7.1 Introduction

This chapter describes the development of the UX/UI integration module, which brings together all platform components into a cohesive, user-friendly interface. The module focuses on creating intuitive user experiences for diverse stakeholders including commuters, transportation planners, and system administrators, ensuring that complex data and functionality are accessible through well-designed interfaces.

Effective UX/UI integration transforms raw data and backend services into actionable insights and seamless user interactions, making the transportation platform valuable and usable.

7.2 Implemented Tasks

The UX/UI implementation integrates all frontend features documented for MAP APP HCMUS into a cohesive, responsive experience. Key deliverables include authentication flows, map and routing interfaces, bus routing UI, styling system, and application structure.

7.2.1 Authentication & OAuth UI

- **Login/Register Pages (`src/pages/Login.js`, `src/pages/Register.js`):** Modern forms with validation, loading/error states, and mobile-first layout. Sidebar is hidden on small screens; the form expands full-screen on mobile.
- **OAuth Buttons (`src/components/OAuthButtons.js`):** Google and GitHub login with branded icons, hover animations, and an “Hoặc” divider for clear choice. Facebook OAuth is disabled (requires Business Verification).
- **OAuth Flow Handling (`src/utils/oauth.js`):** Google SDK initialization and GitHub redirect/callback handling integrated into the UI lifecycle; auto-detects OAuth callback codes and finalizes sign-in.
- **Session UX (`src/utils/tokenManager.js`, `src/api.js`):** Auto-logout on token expiration surfaced with friendly messaging via centralized API error handling.

- **Compliance Page (`public/privacy-policy.html`):** Dedicated privacy policy to support OAuth application requirements.

7.2.2 Map & Routing Interfaces

- **Routes Page (`src/pages/RoutesPage.js`):** Route planning UI with travel mode selector (car, motorcycle, bicycle, pedestrian) and route type toggle (fastest/shortest). Displays distance and duration with real-time recalculation.
- **Search Box (`src/SearchBoxRoutes.js`):** Origin/destination inputs with Goong autocomplete, clear icons, and integrated mode/type controls.
- **SOS Map (`src/GoongSOSMap.js`):** Interactive map with SOS location marking, camera overlays, and user-location tracking.
- **Bus Map Page (`src/pages/BusMapPage.js`):** Bus routing UI that renders routes and nearby stops, integrated with the bus routing API and graph cache.

7.2.3 Styling & Design System

- **Tailwind CSS:** Utility-first styling with project-specific configuration (`tailwind.config.js`, `postcss.config.js`).
- **Auth Styles (`src/css/Auth.css`):** Animated backgrounds (grid, orbs), responsive panels, and polished OAuth buttons (blue gradient for Google, dark hover for GitHub).
- **Global Styles (`src/css/index.css`):** Baseline resets, variables, and utility classes for consistent look-and-feel.
- **Responsiveness:** Mobile-first layout patterns across pages; controls and typography scale appropriately from phones to desktops.
- **Accessibility (baseline):** High-contrast controls and keyboard-focusable inputs; structure prepared for future WCAG improvements.

7.2.4 App Structure & Navigation

- **Routing (`src/App.js`):** Clear application routes: Home, Login, Register, Routes, Bus, and SOS. Supports protected routes via centralized auth state.
- **Centralized API Handling (`src/api.js`):** Uniform error handling for user-friendly feedback (e.g., auto-logout on HTTP 401) and consistent request lifecycles.
- **Performance Considerations:** Optimized API usage and caching strategies at the UI layer to keep interactions responsive.

7.3 Next Steps

Aligned with the frontend roadmap, upcoming UX/UI work will focus on depth, performance, and accessibility:

- **Authentication Enhancements:** Email verification, password reset flows, 2FA, and “Remember me” for improved session UX.
- **Map Features:** Real-time traffic overlays, favorite locations, location sharing, offline map support, and custom map styles.
- **Bus Routing UX:** Multiple route options, transfer optimization, fare calculation, and arrival predictions when data is available.
- **PWA & Notifications:** Progressive Web App capabilities with push notifications for SOS alerts and service updates.
- **Internationalization:** Multi-language UI (e.g., Vietnamese/English) and locale-aware formatting.
- **Accessibility:** Systematic WCAG improvements (focus order, ARIA roles, color-contrast audits, keyboard navigation coverage).
- **Performance:** Service workers, code splitting, image optimization, CDN integration, and refined caching strategies.
- **Analytics:** Privacy-respecting analytics for route popularity and UI performance monitoring.

7.4 Summary

The current UX/UI delivers a modern, mobile-first experience with:

- Seamless Google/GitHub sign-in, robust token handling, and clear error states
- Intuitive route planning with mode/type selectors and real-time results on Goong Maps
- SOS and camera overlays, plus dedicated bus routing visuals with nearby stop discovery
- Tailwind-driven design system, animated yet performant Auth pages, and consistent app navigation

User-facing performance and responsiveness have improved significantly thanks to centralized API handling and UI-side optimizations. Next steps emphasize PWA capabilities, accessibility, internationalization, real-time traffic visualization, and richer bus routing options to further elevate the end-user experience.